# Privacy-Preserving Public Auditing for Secure Cloud Storage

Cong Wang, *Member*, *IEEE*, Sherman S.M. Chow, Qian Wang, *Member*, *IEEE*, Kui Ren, *Senior Member*, *IEEE*, and Wenjing Lou, *Senior Member*, *IEEE*

**Abstract**—Using cloud storage, users can remotely store their data and enjoy the on-demand high-quality applications and services from a shared pool of configurable computing resources, without the burden of local data storage and maintenance. However, the fact that users no longer have physical possession of the outsourced data makes the data integrity protection in cloud computing a formidable task, especially for users with constrained computing resources. Moreover, users should be able to just use the cloud storage as if it is local, without worrying about the need to verify its integrity. Thus, enabling public auditability for cloud storage is of critical importance so that users can resort to a third-party auditor (TPA) to check the integrity of outsourced data and be worry free. To securely introduce an effective TPA, the auditing process should bring in no new vulnerabilities toward user data privacy, and introduce no additional online burden to user. In this paper, we propose a secure cloud storage system supporting privacy-preserving public auditing. We further extend our result to enable the TPA to perform audits for multiple users simultaneously and efficiently. Extensive security and performance analysis show the proposed schemes are provably secure and highly efficient. Our preliminary experiment conducted on Amazon EC2 instance further demonstrates the fast performance of the design.

**Index Terms**—Data storage, privacy preserving, public auditability, cloud computing, delegation, batch verification, zero knowledge

## 1 Introduction

Cloud computing has been envisioned as the next-generation information technology (IT) architecture for enterprises, due to its long list of unprecedented advantages in the IT history: on-demand self-service, ubiquitous network access, location independent resource pooling, rapid resource elasticity, usage-based pricing and transference of risk [2]. As a disruptive technology with profound implications, cloud computing is transforming the very nature of how businesses use information technology. One fundamental aspect of this paradigm shifting is that data are being centralized or outsourced to the cloud. From users' perspective, including both individuals and IT enterprises, storing data remotely to the cloud in a flexible on-demand manner brings appealing benefits: relief of the burden for storage management, universal data access with location independence, and avoidance of capital expenditure on hardware, software, and personnel maintenances, etc., [3].

While cloud computing makes these advantages more appealing than ever, it also brings new and challenging security threats toward users' outsourced data. Since cloud service providers (CSP) are separate administrative entities, data outsourcing is actually relinquishing user's ultimate control over the fate of their data. As a result, the correctness of the data in the cloud is being put at risk due to the following reasons. First of all, although the infrastructures under the cloud are much more powerful and reliable than personal computing devices, they are still facing the broad range of both internal and external threats for data integrity [4]. Examples of outages and security breaches of noteworthy cloud services appear from time to time [5], [6], [7]. Second, there do exist various motivations for CSP to behave unfaithfully toward the cloud users regarding their outsourced data status. For examples, CSP might reclaim storage for monetary reasons by discarding data that have not been or are rarely accessed, or even hide data loss incidents to maintain a reputation [8], [9], [10]. In short, although outsourcing data to the cloud is economically attractive for long-term large-scale storage, it does not immediately offer any guarantee on data integrity and availability. This problem, if not properly addressed, may impede the success of cloud architecture.

As users no longer physically possess the storage of their data, traditional cryptographic primitives for the purpose of data security protection cannot be directly adopted [11]. In particular, simply downloading all the data for its integrity verification is not a practical solution due to the expensiveness in I/O and transmission cost across the network. Besides, it is often insufficient to detect the data corruption only when accessing the data, as it does not give users correctness assurance for those unaccessed data and might be too late to recover the data loss or damage.

- *C. Wang is with the Department of Computer Science, City University of Hong Kong, Hong Kong. E-mail: congwang@cityu.edu.hk.*
- *S.S.M. Chow is with the Department of Information Engineering, Chinese University of Hong Kong, Shatin, N.T., Hong Kong. E-mail: smchow@ie.cuhk.edu.hk.*
- *Q. Wang is with the School of Computer, Wuhan University, China. E-mail: qianwang@whu.edu.cn.*
- *K. Ren is with the Department of Computer Science and Engineering, The State University of New York at Buffalo. E-mail: kuiren@buffalo.edu.*
- *W. Lou is with the Department of Computer Science, Virginia Polytechnic Institute and State University, 7054 Haycock Rd., Falls Church, VA 22043. E-mail: wjlou@vt.edu.*

*Manuscript received 15 Nov. 2010; revised 12 May 2011; accepted 16 Oct. 2011; published online 15 Dec. 2011.*
*Recommended for acceptance by B. Veeravalli.*
*For information on obtaining reprints of this article, please send e-mail to: tc@computer.org, and reference IEEECS Log Number TC-2010-11-0627.*
*Digital Object Identifier no. 10.1109/TC.2011.245.*

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.
0018-9340/13/$31.00 © 2013 IEEE Published by the IEEE Computer Society

Considering the large size of the outsourced data and the user's constrained resource capability, the tasks of auditing the data correctness in a cloud environment can be formidable and expensive for the cloud users [12], [8]. Moreover, the overhead of using cloud storage should be minimized as much as possible, such that a user does not need to perform too many operations to use the data (in additional to retrieving the data). In particular, users may not want to go through the complexity in verifying the data integrity. Besides, there may be more than one user accesses the same cloud storage, say in an enterprise setting. For easier management, it is desirable that cloud only entertains verification request from a single designated party.

To fully ensure the data integrity and save the cloud users' computation resources as well as online burden, it is of critical importance to enable public auditing service for cloud data storage, so that users may resort to an independent third-party auditor (TPA) to audit the outsourced data when needed. The TPA, who has expertise and capabilities that users do not, can periodically check the integrity of all the data stored in the cloud on behalf of the users, which provides a much more easier and affordable way for the users to ensure their storage correctness in the cloud. Moreover, in addition to help users to evaluate the risk of their subscribed cloud data services, the audit result from TPA would also be beneficial for the cloud service providers to improve their cloud-based service platform, and even serve for independent arbitration purposes [10]. In a word, enabling public auditing services will play an important role for this nascent cloud economy to become fully established, where users will need ways to assess risk and gain trust in the cloud.

Recently, the notion of public auditability has been proposed in the context of ensuring remotely stored data integrity under different system and security models [9], [13], [11], [8]. Public auditability allows an external party, in addition to the user himself, to verify the correctness of remotely stored data. However, most of these schemes [9], [13], [8] do not consider the privacy protection of users' data against external auditors. Indeed, they may potentially reveal user's data to auditors, as will be discussed in Section 3.4. This severe drawback greatly affects the security of these protocols in cloud computing. From the perspective of protecting data privacy, the users, who own the data and rely on TPA just for the storage security of their data, do not want this auditing process introducing new vulnerabilities of unauthorized information leakage toward their data security [14], [15]. Moreover, there are legal regulations, such as the US Health Insurance Portability and Accountability Act (HIPAA) [16], further demanding the outsourced data not to be leaked to external parties [10]. Simply exploiting data encryption before outsourcing [15], [11] could be one way to mitigate this privacy concern of data auditing, but it could also be an overkill when employed in the case of unencrypted/public cloud data (e.g., outsourced libraries and scientific data sets), due to the unnecessary processing burden for cloud users. Besides, encryption does not completely solve the problem of protecting data privacy against third-party auditing but just reduces it to the complex key management domain. Unauthorized data leakage still remains possible due to the potential exposure of decryption keys.

Therefore, how to enable a privacy-preserving third-party auditing protocol, independent to data encryption, is the problem we are going to tackle in this paper. Our work is among the first few ones to support privacy-preserving public auditing in cloud computing, with a focus on data storage. Besides, with the prevalence of cloud computing, a foreseeable increase of auditing tasks from different users may be delegated to TPA. As the individual auditing of these growing tasks can be tedious and cumbersome, a natural demand is then how to enable the TPA to efficiently perform multiple auditing tasks in a batch manner, i.e., simultaneously.

To address these problems, our work utilizes the technique of public key-based homomorphic linear authenticator (or HLA for short) [9], [13], [8], which enables TPA to perform the auditing without demanding the local copy of data and thus drastically reduces the communication and computation overhead as compared to the straightforward data auditing approaches. By integrating the HLA with random masking, our protocol guarantees that the TPA could not learn any knowledge about the data content stored in the cloud server (CS) during the efficient auditing process. The aggregation and algebraic properties of the authenticator further benefit our design for the batch auditing. Specifically, our contribution can be summarized as the following three aspects:

1. We motivate the public auditing system of data storage security in cloud computing and provide a privacy-preserving auditing protocol. Our scheme enables an external auditor to audit user's cloud data without learning the data content.
2. To the best of our knowledge, our scheme is the first to support scalable and efficient privacy-preserving public storage auditing in cloud. Specifically, our scheme achieves batch auditing where multiple delegated auditing tasks from different users can be performed simultaneously by the TPA in a privacy-preserving manner.
3. We prove the security and justify the performance of our proposed schemes through concrete experiments and comparisons with the state of the art.

The rest of the paper is organized as follows: Section 2 introduces the system and threat model, and our design goals. Then, we provide the detailed description of our scheme in Section 3. Section 4 gives the security analysis and performance evaluation. Section 5 presents further discussions on a zero-knowledge auditing protocol, followed by Section 6 that overviews the related work. Finally, Section 7 gives the concluding remark of the whole paper.

## 2 Problem Statement

### 2.1 The System and Threat Model

We consider a cloud data storage service involving three different entities, as illustrated in Fig. 1: the *cloud user*, who has large amount of data files to be stored in the cloud; the *cloud server*, which is managed by the *cloud service provider* to provide data storage service and has significant storage

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

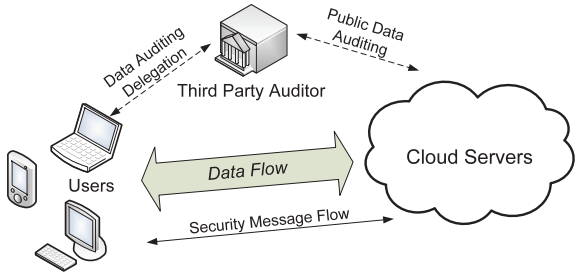



Fig. 1. The architecture of cloud data storage service.

space and computation resources (we will not differentiate CS and CSP hereafter); the *third-party auditor*, who has expertise and capabilities that cloud users do not have and is trusted to assess the cloud storage service reliability on behalf of the user upon request. Users rely on the CS for cloud data storage and maintenance. They may also dynamically interact with the CS to access and update their stored data for various application purposes. As users no longer possess their data locally, it is of critical importance for users to ensure that their data are being correctly stored and maintained. To save the computation resource as well as the online burden potentially brought by the periodic storage correctness verification, cloud users may resort to TPA for ensuring the storage integrity of their outsourced data, while hoping to keep their data private from TPA.

We assume the data integrity threats toward users' data can come from both internal and external attacks at CS. These may include: software bugs, hardware failures, bugs in the network path, economically motivated hackers, malicious or accidental management errors, etc. Besides, CS can be self-interested. For their own benefits, such as to maintain reputation, CS might even decide to hide these data corruption incidents to users. Using third-party auditing service provides a cost-effective method for users to gain trust in cloud. We assume the TPA, who is in the business of auditing, is reliable and independent. However, it may harm the user if the TPA could learn the outsourced data after the audit.

Note that in our model, beyond users' reluctance to leak data to TPA, we also assume that cloud servers has no incentives to reveal their hosted data to external parties. On the one hand, there are regulations, e.g., HIPAA [16], requesting CS to maintain users' data privacy. On the other hand, as users' data belong to their business asset [10], there also exist financial incentives for CS to protect it from any external parties. Therefore, we assume that neither CS nor TPA has motivations to collude with each other during the auditing process. In other words, neither entities will deviate from the prescribed protocol execution in the following presentation.

To authorize the CS to respond to the audit delegated to TPA's, the user can issue a certificate on TPA's public key, and all audits from the TPA are authenticated against such a certificate. These authentication handshakes are omitted in the following presentation.

### 2.2 Design Goals

To enable privacy-preserving public auditing for cloud data storage under the aforementioned model, our protocol design should achieve the following security and performance guarantees:

1. Public auditability: to allow TPA to verify the correctness of the cloud data on demand without retrieving a copy of the whole data or introducing additional online burden to the cloud users.
2. Storage correctness: to ensure that there exists no cheating cloud server that can pass the TPA's audit without indeed storing users' data intact.
3. Privacy preserving: to ensure that the TPA cannot derive users' data content from the information collected during the auditing process.
4. Batch auditing: to enable TPA with secure and efficient auditing capability to cope with multiple auditing delegations from possibly large number of different users simultaneously.
5. Lightweight: to allow TPA to perform auditing with minimum communication and computation overhead.

## 3 The Proposed Schemes

This section presents our public auditing scheme which provides a *complete outsourcing* solution of data—not only the data itself, but also its integrity checking. After introducing notations and brief preliminaries, we start from an overview of our public auditing system and discuss two straightforward schemes and their demerits. Then, we present our main scheme and show how to extent our main scheme to support batch auditing for the TPA upon delegations from multiple users. Finally, we discuss how to generalize our privacy-preserving public auditing scheme and its support of data dynamics.

### 3.1 Notation and Preliminaries

- $F$—the data file to be outsourced, denoted as a sequence of $n$ blocks $m_1, \ldots, m_i, \ldots, m_n \in \mathbb{Z}_p$ for some large prime $p$.
- $MAC_{(\cdot)}(\cdot)$—message authentication code (MAC) function, defined as: $\mathcal{K} \times \{0,1\}^* \rightarrow \{0,1\}^l$ where $\mathcal{K}$ denotes the key space.
- $H(\cdot)$, $h(\cdot)$—cryptographic hash functions.

We now introduce some necessary cryptographic background for our proposed scheme.

*Bilinear Map.* Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be multiplicative cyclic groups of prime order $p$. Let $g_1$ and $g_2$ be generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. A bilinear map is a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ such that for all $u \in \mathbb{G}_1$, $v \in \mathbb{G}_2$ and $a, b \in \mathbb{Z}_p$, $e(u^a, v^b) = e(u, v)^{ab}$. This bilinearity implies that for any $u_1$, $u_2 \in \mathbb{G}_1$, $v \in \mathbb{G}_2$, $e(u_1 \cdot u_2, v) = e(u_1, v) \cdot e(u_2, v)$. Of course, there exists an efficiently computable algorithm for computing $e$ and the map should be nontrivial, i.e., $e$ is nondegenerate: $e(g_1, g_2) \neq 1$.

### 3.2 Definitions and Framework

We follow a similar definition of previously proposed schemes in the context of remote data integrity checking [9], [11], [13] and adapt the framework for our privacy-preserving public auditing system.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

A public auditing scheme consists of four algorithms (KeyGen, SigGen, GenProof, VerifyProof). KeyGen is a key generation algorithm that is run by the user to setup the scheme. SigGen is used by the user to generate verification metadata, which may consist of digital signatures. GenProof is run by the cloud server to generate a proof of data storage correctness, while VerifyProof is run by the TPA to audit the proof.

Running a public auditing system consists of two phases, Setup and Audit:

- Setup: The user initializes the public and secret parameters of the system by executing KeyGen, and preprocesses the data file $F$ by using SigGen to generate the verification metadata. The user then stores the data file $F$ and the verification metadata at the cloud server, and deletes its local copy. As part of preprocessing, the user may alter the data file $F$ by expanding it or including additional metadata to be stored at server.
- Audit: The TPA issues an audit message or challenge to the cloud server to make sure that the cloud server has retained the data file $F$ properly at the time of the audit. The cloud server will derive a response message by executing GenProof using $F$ and its verification metadata as inputs. The TPA then verifies the response via VerifyProof.

Our framework assumes that the TPA is stateless, i.e., TPA does not need to maintain and update state between audits, which is a desirable property especially in the public auditing system [13]. Note that it is easy to extend the framework above to capture a stateful auditing system, essentially by splitting the verification metadata into two parts which are stored by the TPA and the cloud server, respectively. Our design does not assume any additional property on the data file. If the user wants to have more error resilience, he can first redundantly encodes the data file and then uses our system with the data that has error-correcting codes integrated.[1]

### 3.3 The Basic Schemes

Before giving our main result, we study two classes of schemes as a warmup. The first one is a MAC-based solution which suffers from undesirable systematic demerits—bounded usage and stateful verification, which may pose additional online burden to users, in a public auditing setting. This also shows that the auditing problem is still not easy to solve even if we have introduced a TPA. The second one is a system based on homomorphic linear authenticators, which covers many recent proof of storage systems. We will pinpoint the reason why all existing HLA-based systems are not privacy preserving. The analysis of these basic schemes leads to our main result, which overcomes all these drawbacks. Our main scheme to be presented is based on a specific HLA scheme.

**MAC-based solution.** There are two possible ways to make use of MAC to authenticate the data. A trivial way is just uploading the data blocks with their MACs to the server, and sends the corresponding secret key $sk$ to the TPA. Later, the TPA can randomly retrieve blocks with their MACs and check the correctness via $sk$. Apart from the high (linear in the sampled data size) communication and computation complexities, the TPA requires the knowledge of the data blocks for verification.

To circumvent the requirement of the data in TPA verification, one may restrict the verification to just consist of equality checking. The idea is as follows: Before data outsourcing, the cloud user chooses $s$ random message authentication code keys $\{sk_\tau\}_{1\le\tau\le s}$, precomputes $s$ (deterministic) MACs, $\{MAC_{sk_\tau}(F)\}_{1\le\tau\le s}$ for the whole data file $F$, and publishes these verification metadata (the keys and the MACs) to TPA. The TPA can reveal a secret key $sk_\tau$ to the cloud server and ask for a fresh keyed MAC for comparison in each audit. This is privacy preserving as long as it is impossible to recover $F$ in full given $MAC_{sk_\tau}(F)$ and $sk_\tau$. However, it suffers from the following severe drawbacks: 1) the number of times a particular data file can be audited is limited by the number of secret keys that must be fixed a priori. Once all possible secret keys are exhausted, the user then has to retrieve data in full to recompute and republish new MACs to TPA; 2) The TPA also has to maintain and update state between audits, i.e., keep track on the revealed MAC keys. Considering the potentially large number of audit delegations from multiple users, maintaining such states for TPA can be difficult and error prone; 3) it can only support static data, and cannot efficiently deal with dynamic data at all. However, supporting data dynamics is also of critical importance for cloud storage systems. For the reason of brevity and clarity, our main protocol will be presented based on static data. Section 3.6 will describe how to adapt our protocol for dynamic data.

**HLA-based solution.** To effectively support public auditability without having to retrieve the data blocks themselves, the HLA technique [9], [13], [8] can be used. HLAs, like MACs, are also some unforgeable verification metadata that authenticate the integrity of a data block. The difference is that HLAs can be aggregated. It is possible to compute an aggregated HLA which authenticates a linear combination of the individual data blocks.

At a high level, an HLA-based proof of storage system works as follow. The user still authenticates each element of $F = \{m_i\}$ by a set of HLAs $\Phi$. The TPA verifies the cloud storage by sending a random set of challenge $\{\nu_i\}$. The cloud server then returns $\mu = \sum_i \nu_i \cdot m_i$ and its aggregated authenticator $\sigma$ computed from $\Phi$.

Though allowing efficient data auditing and consuming only constant bandwidth, the direct adoption of these HLA-based techniques is still not suitable for our purposes. This is because the linear combination of blocks, $\mu = \sum_i \nu_i \cdot m_i$, may potentially reveal user data information to TPA, and violates the privacy-preserving guarantee. Specifically, by challenging the same set of $c$ block $m_1, m_2, \ldots, m_c$ using $c$ different sets of random coefficients $\{\nu_i\}$, TPA can accumulate $c$ different linear combinations $\mu_1, \ldots, \mu_c$. With $\{\mu_i\}$ and $\{\nu_i\}$, TPA can derive the user's data $m_1, m_2, \ldots, m_c$ by simply solving a system of linear equations.

1. We refer readers to [17], [18] for the details on integration of error-correcting codes and remote data integrity checking.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
The Privacy-Preserving Public Auditing Protocol

| TPA | | Cloud Server |
|---|---|---|
| 1. Retrieve file tag $t$, verify its signature, and quit if fail; | | |
| 2. Generate a random challenge $chal = \{(i, \nu_i)\}_{i \in I}$; | $\xrightarrow[\text{challenge request } chal]{\{(i,\nu_i)\}_{i\in I}}$ | 3. Compute $\mu' = \sum_{i\in I} \nu_i m_i$, and $\sigma = \prod_{i\in I} \sigma_i^{\nu_i}$; 4. Randomly pick $r \leftarrow \mathbb{Z}_p$, and $R = e(u,v)^r$ and $\gamma = h(R)$; |
| | $\xleftarrow[\text{storage correctness proof}]{\{\mu,\sigma,R\}}$ | 5. Compute $\mu = r + \gamma\mu' \mod p$ ; |
| 6. Compute $\gamma = h(R)$, and then verify $\{\mu, \sigma, R\}$ via Equation 1. | | |

### 3.4 Privacy-Preserving Public Auditing Scheme

**Overview.** To achieve privacy-preserving public auditing, we propose to uniquely integrate the homomorphic linear authenticator with random masking technique. In our protocol, the linear combination of sampled blocks in the server's response is masked with randomness generated by the server. With random masking, the TPA no longer has all the necessary information to build up a correct group of linear equations and therefore cannot derive the user's data content, no matter how many linear combinations of the same set of file blocks can be collected. On the other hand, the correctness validation of the block-authenticator pairs can still be carried out in a new way which will be shown shortly, even with the presence of the randomness. Our design makes use of a public key-based HLA, to equip the auditing protocol with public auditability. Specifically, we use the HLA proposed in [13], which is based on the short signature scheme proposed by Boneh, Lynn, and Shacham (hereinafter referred as BLS signature) [19].

**Scheme details.** Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be multiplicative cyclic groups of prime order $p$, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear map as introduced in preliminaries. Let $g$ be a generator of $\mathbb{G}_2$. $H(\cdot)$ is a secure map-to-point hash function: $\{0,1\}^* \rightarrow \mathbb{G}_1$, which maps strings uniformly to $\mathbb{G}_1$. Another hash function $h(\cdot) : \mathbb{G}_T \rightarrow \mathbb{Z}_p$ maps group element of $\mathbb{G}_T$ uniformly to $\mathbb{Z}_p$. Our scheme is as follows:

`Setup` Phase: The cloud user runs `KeyGen` to generate the public and secret parameters. Specifically, the user chooses a random signing key pair $(spk, ssk)$, a random $x \leftarrow \mathbb{Z}_p$, a random element $u \leftarrow \mathbb{G}_1$, and computes $v \leftarrow g^x$. The secret parameter is $sk = (x, ssk)$ and the public parameters are $pk = (spk, v, g, u, e(u, v))$.

Given a data file $F = \{m_i\}$, the user runs `SigGen` to compute authenticator $\sigma_i \leftarrow (H(W_i) \cdot u^{m_i})^x \in \mathbb{G}_1$ for each $i$. Here, $W_i = name \| i$ and $name$ is chosen by the user uniformly at random from $\mathbb{Z}_p$ as the identifier of file $F$. Denote the set of authenticators by $\Phi = \{\sigma_i\}_{1 \le i \le n}$.

The last part of `SigGen` is for ensuring the integrity of the unique file identifier $name$. One simple way to do this is to compute $t = name \| \mathrm{SSig}_{ssk}(name)$ as the file tag for $F$, where $\mathrm{SSig}_{ssk}(name)$ is the signature on $name$ under the private key $ssk$. For simplicity, we assume the TPA knows the number of blocks $n$. The user then sends $F$ along with the verification metadata $(\Phi, t)$ to the server and deletes them from local storage.

`Audit` Phase: The TPA first retrieves the file tag $t$. With respect to the mechanism we describe in the `Setup` phase, the TPA verifies the signature $\mathrm{SSig}_{ssk}(name)$ via $spk$, and quits by emitting FALSE if the verification fails. Otherwise, the TPA recovers $name$.

Now it comes to the "core" part of the auditing process. To generate the challenge message for the audit "$chal$," the TPA picks a random $c$-element subset $I = \{s_1, \ldots, s_c\}$ of set $[1, n]$. For each element $i \in I$, the TPA also chooses a random value $\nu_i$ (of bit length that can be shorter than $|p|$, as explained in [13]). The message "$chal$" specifies the positions of the blocks required to be checked. The TPA sends $chal = \{(i, \nu_i)\}_{i \in I}$ to the server.

Upon receiving challenge $chal = \{(i, \nu_i)\}_{i \in I}$, the server runs `GenProof` to generate a response proof of data storage correctness. Specifically, the server chooses a random element $r \leftarrow \mathbb{Z}_p$, and calculates $R = e(u, v)^r \in \mathbb{G}_T$. Let $\mu'$ denote the linear combination of sampled blocks specified in $chal$: $\mu' = \sum_{i \in I} \nu_i m_i$. To blind $\mu'$ with $r$, the server computes: $\mu = r + \gamma\mu' \bmod p$, where $\gamma = h(R) \in \mathbb{Z}_p$. Meanwhile, the server also calculates an aggregated authenticator $\sigma = \prod_{i \in I} \sigma_i^{\nu_i} \in \mathbb{G}_1$. It then sends $\{\mu, \sigma, R\}$ as the response proof of storage correctness to the TPA. With the response, the TPA runs `VerifyProof` to validate it by first computing $\gamma = h(R)$ and then checking the verification equation

$$R \cdot e(\sigma^\gamma, g) \stackrel{?}{=} e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^\gamma \cdot u^\mu, v\right). \tag{1}$$

The protocol is illustrated in Table 1. The correctness of the above verification equation is elaborated as follows:

$$\begin{aligned} R \cdot e(\sigma^\gamma, g) &= e(u,v)^r \cdot e\left(\left(\prod_{i=s_1}^{s_c} (H(W_i) \cdot u^{m_i})^{x \cdot \nu_i}\right)^\gamma, g\right) \\ &= e(u^r, v) \cdot e\left(\left(\prod_{i=s_1}^{s_c} (H(W_i)^{\nu_i} \cdot u^{\nu_i m_i}\right)^\gamma, g\right)^x \\ &= e(u^r, v) \cdot e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^\gamma \cdot u^{\mu'\gamma}, v\right) \\ &= e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^\gamma \cdot u^{\mu'\gamma + r}, v\right) \\ &= e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^\gamma \cdot u^{\mu}, v\right). \end{aligned}$$

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

**Properties of our protocol.** It is easy to see that our protocol achieves public auditability. There is no secret keying material or states for the TPA to keep or maintain between audits, and the auditing protocol does not pose any potential online burden on users. This approach ensures the privacy of user data content during the auditing process by employing a random masking $r$ to hide $\mu$, a linear combination of the data blocks. Note that the value $R$ in our protocol, which enables the privacy-preserving guarantee, will not affect the validity of the equation, due to the circular relationship between $R$ and $\gamma$ in $\gamma = h(R)$ and the verification equation. Storage correctness thus follows from that of the underlying protocol [13]. The security of this protocol will be formally proven in Section 4. Besides, the HLA helps achieve the constant communication overhead for server's response during the audit: the size of $\{\sigma, \mu, R\}$ is independent of the number of sampled blocks $c$.

Previous work [9], [8] showed that if the server is missing a fraction of the data, then the number of blocks that needs to be checked in order to detect server misbehavior with high probability is in the order of $O(1)$. In particular, if $t$ fraction of data is corrupted, then random sampling $c$ blocks would reach the detection probability $P = 1 - (1-t)^c$. Here, every block is chosen uniformly at random. When $t = 1\%$ of the data $F$, the TPA only needs to audit for $c = 300$ or 460 randomly chosen blocks of $F$ to detect this misbehavior with probability larger than 95 and 99 percent, respectively. Given the huge volume of data outsourced in the cloud, checking a portion of the data file is more affordable and practical for both the TPA and the cloud server than checking all the data, as long as the sampling strategies provides high-probability assurance. In Section 4, we will present the experiment result based on these sampling strategies.

For some cloud storage providers, it is possible that certain information dispersal algorithms (IDA) may be used to fragment and geographically distribute the user's outsourced data for increased availability. We note that these cloud side operations would not affect the behavior of our proposed mechanism, as long as the IDA is systematic, i.e., it preserves user's data in its original form after encoding with redundancy. This is because from user's perspective, as long as there is a complete yet unchanged copy of his outsourced data in cloud, the precomputed verification metadata $(\Phi, t)$ will remain valid. As a result, those metadata can still be utilized in our auditing mechanism to guarantee the correctness of user's outsourced cloud data.

**Storage and communication tradeoff.** As described above, each block is accompanied by an authenticator of equal size of $|p|$ bits. This gives about $2\times$ storage overhead on server. However, as noted in [13], we can introduce a parameter $s$ in the authenticator construction to adjust this storage overhead, in the cost of communication overhead in the auditing protocol between TPA and cloud server. In particular, we assume each block $m_i$ consists of $s$ sectors $\{m_{ij}\}$ with $1 \le j \le s$, where $m_{ij} \in \mathbb{Z}_p$. The public parameter $pk$ is now $(spk, v, g, \{u_j\}, \{e(u_j, v)\})$, $1 \le j \le s$, where $u_1, u_2, \ldots, u_s$ are randomly chosen from $\mathbb{G}_1$. The authenticator $\sigma_i$ of $m_i$ is constructed as: $\sigma_i \leftarrow (H(W_i) \cdot \prod_{j=1}^{s} u_j^{m_{ij}})^x \in \mathbb{G}_1$. Because we now have one authenticator per block (or per $s$ sectors), we reduce the storage overhead to $(1 + 1/s)\times$.

To respond to the auditing challenge $chal = \{(i, \nu_i)\}_{i \in I}$, for $1 \le j \le s$, the cloud server chooses a random elements $r_j \leftarrow \mathbb{Z}_p$, and calculates $R_j = e(u, v)^{r_j} \in \mathbb{G}_T$. Then, the server blinds each $\mu'_j = \sum_{i \in I} \nu_i m_{ij}$ with $r_j$, and derives the blinded $\mu_j = r_j + \gamma \mu'_j \bmod p$, where $\gamma = h(R_1 \| R_2 \| \cdots \| R_s) \in \mathbb{Z}_p$. The aggregated authenticator is still computed as before. It then sends $\{\sigma, \{\mu_j, R_j\}_{1 \le j \le s}\}$ as the proof response to TPA. With the proof, TPA first computes $\gamma = h(R_1 \| R_2 \| \cdots \| R_s)$, and then checks the following verification:

$$R_1 \cdots R_s \cdot e(\sigma^\gamma, g) \stackrel{?}{=} e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^\gamma \cdot \prod_{j=1}^{s} u_j^{\mu_j}, v\right). \quad (2)$$

The correctness elaboration is similar to (1) and thus omitted. The overall storage overhead is reduced to $(1 + 1/s)\times$, but the proof size now increases roughly $s\times$ due to the additional $s$ element pairs $\{\mu_j, R_j\}_{1 \le j \le s}$ that the cloud server has to return. For presentation simplicity, we continue to choose $s = 1$ in our following scheme description. We will present some experiment results with larger choice of $s$ in Section 4.

### 3.5 Support for Batch Auditing

With the establishment of privacy-preserving public auditing, the TPA may concurrently handle multiple auditing upon different users' delegation. The individual auditing of these tasks for the TPA can be tedious and very inefficient. Given $K$ auditing delegations on $K$ distinct data files from $K$ different users, it is more advantageous for the TPA to batch these multiple tasks together and audit at one time. Keeping this natural demand in mind, we slightly modify the protocol in a single user case, and achieves the aggregation of $K$ verification equations (for $K$ auditing tasks) into a single one, as shown in (3). As a result, a secure batch auditing protocol for simultaneous auditing of multiple tasks is obtained. The details are described as follows:

`Setup` phase: Basically, the users just perform `Setup` independently. Suppose there are $K$ users in the system, and each user $k$ has a data file $F_k = (m_{k,1}, \ldots, m_{k,n})$ to be outsourced to the cloud server, where $k \in \{1, \ldots, K\}$. For simplicity, we assume each file $F_k$ has the same number of $n$ blocks. For a particular user $k$, denote his/her secret key as $(x_k, ssk_k)$, and the corresponding public parameter as $(spk_k, v_k, g, u_k, e(u_k, v_k))$ where $v_k = g^{x_k}$. Similar to the single user case, each user $k$ has already randomly chosen a different (with overwhelming probability) name $name_k \in \mathbb{Z}_p$ for his/her file $F_k$, and has correctly generated the corresponding file tag $t_k = name_k \| \mathrm{SSig}_{ssk_k}(name_k)$. Then, each user $k$ runs `SigGen` and computes $\sigma_{k,i}$ for block

$$m_{k,i} : \sigma_{k,i} \leftarrow \left(H(name_k \| i) \cdot u_k^{m_{k,i}}\right)^{x_k} = \left(H(W_{k,i}) \cdot u_k^{m_{k,i}}\right)^{x_k} \in \mathbb{G}_1 (i \in \{1, \ldots, n\}),$$

where $W_{k,i} = name_k \| i$. Finally, each user $k$ sends file $F_k$, set of authenticators $\Phi_k$, and tag $t_k$ to the server and deletes them from local storage.

`Audit` phase: TPA first retrieves and verifies file tag $t_k$ for each user $k$ for later auditing. If the verification fails, TPA quits by emitting FALSE. Otherwise, TPA recovers

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
The Batch Auditing Protocol

| TPA | | Cloud Server |
|---|---|---|
| 1. Verify file tag $t_k$ for each user $k$, and quit if fail; | | For each user $k$ ($1 \le k \le K$): |
| 2. Generate a random challenge $chal = \{(i, \nu_i)\}_{i \in I}$; | $\xrightarrow[\text{challenge request } chal]{\{(i,\nu_i)\}_{i\in I}}$ | 3. Compute $\mu'_k, \sigma_k$, $R_k$ as single user case; |
| | | 4. Compute $\mathcal{R} = R_1 \cdot R_2 \cdots R_K$, $\mathcal{L} = vk_1 \| vk_2 \| \cdots \| vk_K$ and $\gamma_k = h(\mathcal{R} \| v_k \| \mathcal{L})$; |
| | $\xleftarrow[\text{storage correctness proof}]{\{\{\sigma_k,\mu_k\}_{1\le k\le K},\mathcal{R}\}}$ | |
| 6. Compute $\gamma_k = h(\mathcal{R} \| v_k \| \mathcal{L})$ for each user $k$ and do batch auditing via Equation 3. | | 5. Compute $\mu_k = r_k + \gamma_k \mu'_k \mod p$ ; |

$name_k$ and sends the audit challenge $chal = \{(i, \nu_i)\}_{i \in I}$ to the server for auditing data files of all $K$ users.

Upon receiving $chal$, for each user $k \in \{1, \ldots, K\}$, the server randomly picks $r_k \in \mathbb{Z}_p$ and computes $R_k = e(u_k, v_k)^{r_k}$. Denote $\mathcal{R} = R_1 \cdot R_2 \cdots R_K$, and $\mathcal{L} = vk_1 \| vk_2 \| \cdots \| vk_K$, our protocol further requires the server to compute $\gamma_k = h(\mathcal{R} \| v_k \| \mathcal{L})$. Then, the randomly masked responses can be generated as follows:

$$\mu_k = \gamma_k \sum_{i=s_1}^{s_c} \nu_i m_{k,i} + r_k \mod p \text{ and } \sigma_k = \prod_{i=s_1}^{s_c} \sigma_{k,i}^{\nu_i}.$$

The server then responds with $\{\{\sigma_k, \mu_k\}_{1 \le k \le K}, \mathcal{R}\}$.

To verify the response, the TPA can first compute $\gamma_k = h(\mathcal{R} \| v_k \| \mathcal{L})$ for $1 \le k \le K$. Next, TPA checks if the following equation holds:

$$\mathcal{R} \cdot e\left(\prod_{k=1}^{K} \sigma_k^{\gamma_k}, g\right) \stackrel{?}{=} \prod_{k=1}^{K} e\left(\left(\prod_{i=s_1}^{s_c} H(W_{k,i})^{\nu_i}\right)^{\gamma_k} \cdot u_k^{\mu_k}, v_k\right). \quad (3)$$

The batch protocol is illustrated in Table 2. Here, the left-hand side (LHS) of (3) expands as

$$\begin{aligned} \text{LHS} &= R_1 \cdot R_2 \cdots R_K \cdot \prod_{k=1}^{K} e\big(\sigma_k^{\gamma_k}, g\big) \\ &= \prod_{k=1}^{K} R_k \cdot e\big(\sigma_k^{\gamma_k}, g\big) \\ &= \prod_{k=1}^{K} e\left(\left(\prod_{i=s_1}^{s_c} H(W_{k,i})^{\nu_i}\right)^{\gamma_k} \cdot u_k^{\mu_k}, v_k\right), \end{aligned}$$

which is the right-hand side, as required. Note that the last equality follows from (1).

**Efficiency improvement.** As shown in (3), batch auditing not only allows TPA to perform the multiple auditing tasks simultaneously, but also greatly reduces the computation cost on the TPA side. This is because aggregating $K$ verification equations into one helps reduce the number of relatively expensive pairing operations from $2K$, as required in the individual auditing, to $K + 1$, which saves a considerable amount of auditing time.

**Identification of invalid responses.** The verification equation (3) only holds when all the responses are valid, and fails with high probability when there is even one single invalid response in the batch auditing, as we will show in Section 4. In many situations, a response collection may contain invalid responses, especially $\{\mu_k\}_{1 \le k \le K}$, caused by accidental data corruption, or possibly malicious activity by a cloud server. The ratio of invalid responses to the valid could be quite small, and yet a standard batch auditor will reject the entire collection. To further sort out these invalid responses in the batch auditing, we can utilize a recursive divide-and-conquer approach (binary search), as suggested by Ferrara et al. [20]. Specifically, if the batch auditing fails, we can simply divide the collection of responses into two halves, and repeat the auditing on halves via (3). TPA may now require the server to send back all the $\{R_k\}_{1 \le k \le K}$, as in individual auditing. In Section 4.2.2, we show through carefully designed experiment that using this recursive binary search approach, even if up to 20 percent of responses are invalid, batch auditing still performs faster than individual verification.

### 3.6 Support for Data Dynamics

In cloud computing, outsourced data might not only be accessed but also updated frequently by users for various application purposes [21], [8], [22], [23]. Hence, supporting data dynamics for privacy-preserving public auditing is also of paramount importance. Now, we show how to build upon the existing work [8] and adapt our main scheme to support data dynamics, including block level operations of modification, deletion, and insertion.

In [8], data dynamics support is achieved by replacing the index information $i$ with $m_i$ in the computation of block authenticators and using the classic data structure—Merkle hash tree (MHT) [24] for the underlying block sequence enforcement. As a result, the authenticator for each block is changed to $\sigma_i = (H(m_i) \cdot u^{m_i})^x$. We can adopt this technique in our design to achieve privacy-preserving public auditing with support of data dynamics. Specifically, in the `Setup` phase, the user has to generate and send the tree root $TR_{MHT}$ to TPA as additional metadata, where the leaf nodes of MHT are values of $H(m_i)$. In the `Audit` phase, besides $\{\mu, \sigma, R\}$, the server's response should also include $\{H(m_i)\}_{i \in I}$ and their corresponding auxiliary authentication information $aux$ in the MHT. Upon receiving the response, TPA should first use $TR_{MHT}$ and $aux$ to authenticate $\{H(m_i)\}_{i \in I}$ computed by the server. Once $\{H(m_i)\}_{i \in I}$ are authenticated, TPA can then perform the auditing on $\{\mu, \sigma, R, \{H(m_i)\}_{i \in I}\}$ via (1), where $\prod_{s_1 \le i \le s_c} H(W_i)^{\nu_i}$ is now replaced by $\prod_{s_1 \le i \le s_c} H(m_i)^{\nu_i}$. All

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

these changes does not interfere with the proposed random masking technique, so data privacy is still preserved. To support data dynamics, each data update would require the user to generate a new tree root $TR_{MHT}$, which is later sent to TPA as the new metadata for storage auditing task. The details of handling dynamic operations are similar to [8] and thus omitted.

**Application to version control system.** The above scheme allows TPA to always keep the new tree root for auditing the updated data file. But it is worth noting that our mechanism can be easily extended to work with version control system, where both current and previous versions of the data file $F$ and the corresponding authenticators are stored and need to be audited on demand. One possible way is to require TPA to keep tracks of both the current and previous tree roots generated by the user, denoted as $\{TR^1_{MHT}, TR^2_{MHT}, \ldots, TR^V_{MHT}\}$. Here, $V$ is the number of file versions and $TR^V_{MHT}$ is the root related to the most current version of the data file $F$. Then, whenever a designated version $v$ ($1 \leq v \leq V$) of data file is to be audited, the TPA just uses the corresponding $TR^v_{MHT}$ to perform the auditing. The cloud server should also keep track of all the versions of data file $F$ and their authenticators, in order to correctly answer the auditing request from TPA. Note that cloud server does not need to replicate every block of data file in every version, as many of them are the same after updates. However, how to efficiently manage such block storage in cloud is not within the scope of our paper.

### 3.7 Generalization

As mentioned before, our protocol is based on the HLA in [13]. It has been shown in [25] that HLA can be constructed by homomorphic identification protocols. One may apply the random masking technique we used to construct the corresponding zero knowledge proof for different homomorphic identification protocols. Therefore, our privacy-preserving public auditing system for secure cloud storage can be generalized based on other complexity assumptions, such as factoring [25].

## 4 Evaluation

### 4.1 Security Analysis

We evaluate the security of the proposed scheme by analyzing its fulfillment of the security guarantee described in Section 2.2, namely, the storage correctness and privacy-preserving property. We start from the single user case, where our main result is originated. Then, we show the security guarantee of batch auditing for the TPA in multiuser setting.

#### 4.1.1 Storage Correctness Guarantee

We need to prove that the cloud server cannot generate valid response for the TPA without faithfully storing the data, as captured by Theorem 1.

**Theorem 1.** *If the cloud server passes the* `Audit` *phase, it must indeed possess the specified data intact as it is.*

**Proof.** We show that there exists an extractor of $\mu'$ in the random oracle model. With valid $\{\sigma, \mu'\}$, our theorem follows from [13, Theorem 4.2].

The extractor controls the random oracle $h(\cdot)$ and answers the hash query issued by the cloud server, which is treated as an adversary here. For a challenge $\gamma = h(R)$ returned by the extractor, the cloud server outputs $\{\sigma, \mu, R\}$ such that the following equation holds:

$$R \cdot e(\sigma^{\gamma}, g) = e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{\gamma} \cdot u^{\mu}, v\right). \tag{4}$$

Suppose that our extractor can rewind a cloud server in the execution of the protocol to the point just before the challenge $h(R)$ is given. Now, the extractor sets $h(R)$ to be $\gamma^* \neq \gamma$. The cloud server outputs $\{\sigma, \mu^*, R\}$ such that the following equation holds:

$$R \cdot e(\sigma^{\gamma^*}, g) = e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{\gamma^*} \cdot u^{\mu^*}, v\right). \tag{5}$$

The extractor then obtains $\{\sigma, \mu' = (\mu - \mu^*)/(\gamma - \gamma^*)\}$ as a valid response of the underlying proof of storage system [13]. To see why, recall that $\sigma_i = (H(W_i) \cdot u^{m_i})^x$. If we divide (4) by (5), we have

$$e(\sigma^{\gamma-\gamma^*}, g) = e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{\gamma-\gamma^*} \cdot u^{\mu-\mu^*}, v\right)$$

$$e(\sigma^{\gamma-\gamma^*}, g) = e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{\gamma-\gamma^*}, g^x\right) e(u^{\mu-\mu^*}, g^x)$$

$$\sigma^{\gamma-\gamma^*} = \left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{x(\gamma-\gamma^*)} \cdot u^{x(\mu-\mu^*)}$$

$$\left(\prod_{i=s_1}^{s_c} \sigma_i^{\nu_i}\right)^{\gamma-\gamma^*} = \left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{x(\gamma-\gamma^*)} \cdot u^{x(\mu-\mu^*)}$$

$$u^{x(\mu-\mu^*)} = \left(\prod_{i=s_1}^{s_c} (\sigma_i / H(W_i)^x)^{\nu_i}\right)^{\gamma-\gamma^*}$$

$$u^{x(\mu-\mu^*)} = \left(\prod_{i=s_1}^{s_c} (u^{xm_i})^{\nu_i}\right)^{\gamma-\gamma^*}$$

$$\mu - \mu^* = \left(\sum_{i=s_1}^{s_c} m_i \nu_i\right) \cdot (\gamma - \gamma^*)$$

$$\left(\sum_{i=s_1}^{s_c} m_i \nu_i\right) = (\mu - \mu^*)/(\gamma - \gamma^*).$$

Finally, we remark that this extraction argument and the random oracle paradigm are also used in the proof of the underlying scheme [13]. □

#### 4.1.2 Privacy-Preserving Guarantee

The below theorem shows that TPA cannot derive users' data from the information collected during auditing.

**Theorem 2.** *From the server's response $\{\sigma, \mu, R\}$, TPA cannot recover $\mu'$.*

**Proof.** We show the existence of a simulator that can produce a valid response even without the knowledge of $\mu'$, in the random oracle model. Now, the TPA is treated as an adversary. Given a valid $\sigma$ from the

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

cloud server, first, randomly pick $\gamma, \mu$ from $\mathbb{Z}_p$, set $R \leftarrow e((\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i})^{\gamma} \cdot u^{\mu}, v)/e(\sigma^{\gamma}, g)$. Finally, backpatch $\gamma = h(R)$ since the simulator is controlling the random oracle $h(\cdot)$. We remark that this backpatching technique in the random oracle model is also used in the proof of the underlying scheme [13]. □

#### 4.1.3 Security Guarantee for Batch Auditing

Now, we show that our way of extending our result to a multiuser setting will not affect the aforementioned security insurance, as shown in Theorem 3.

**Theorem 3.** *Our batch auditing protocol achieves the same storage correctness and privacy-preserving guarantee as in the single-user case.*

**Proof.** The privacy-preserving guarantee in the multiuser setting is very similar to that of Theorem 2, and thus omitted here. For the storage correctness guarantee, we are going to reduce it to the single-user case. We use the forking technique as in the proof of Theorem 1. However, the verification equation for the batch audits involves $K$ challenges from the random oracle. This time we need to ensure that all the other $K-1$ challenges are determined before the forking of the concerned random oracle response. This can be done using the idea in [26]. As soon as the adversary issues the very first random oracle query for $\gamma_i = h(\mathcal{R}\|v_i\|\mathcal{L})$ for any $i \in [1, K]$, the simulator immediately determines the values $\gamma_j = h(\mathcal{R}\|v_j\|\mathcal{L})$ for all $j \in [1, K]$. This is possible since they are all using the same $\mathcal{R}$ and $\mathcal{L}$. Now, all but one of the $\gamma_k$'s in (3) are equal, so a valid response can be extracted similar to the single-user case in the proof of Theorem 1.□

### 4.2 Performance Analysis

We now report some performance results of our experiments. We consider our auditing mechanism happens between a dedicated TPA and some cloud storage node, where user's data are outsourced to. In our experiment, the TPA/user side process is implemented on a workstation with an Intel Core 2 processor running at 1.86 GHz, 2,048 MB of RAM, and a 7,200 RPM Western Digital 250 GB Serial ATA drive. The cloud server side process is implemented on Amazon Elastic Computing Cloud (EC2) with a large instance type [27], which has 4 EC2 Compute Units, 7.5 GB memory, and 850 GB instance storage. The randomly generated test data is of 1 GB size. All algorithms are implemented using C language. Our code uses the Pairing-Based Cryptography (PBC) library version 0.4.21. The elliptic curve utilized in the experiment is an MNT curve, with base field size of 159 bits and the embedding degree 6. The security level is chosen to be 80 bit, which means $|\nu_i| = 80$ and $|p| = 160$. All experimental results represent the mean of 20 trials.

Because the cloud is a pay-per-use model, users have to pay both the storage cost and the bandwidth cost (for data transfer) when using the cloud storage auditing. Thus, when implementing our mechanism, we have to take into consideration both factors. In particular, we conducts the experiment with two different sets of storage/communication tradeoff parameter $s$ as introduced in Section 3.4. When $s = 1$, the mechanism incurs extra storage cost as large as the data itself, but only takes very small auditing bandwidth cost. Such a mechanism can be adopted when the auditing has to happen very frequently (e.g., checking the storage correctness every few minutes [21]), because the resulting data transfer charge could be dominant in the pay-per-use-model. On the other hand, we also choose a properly larger $s = 10$, which reduces the extra storage cost to only 10 percent of the original data but increases the auditing bandwidth cost roughly 10 times larger than the choice of $s = 1$. Such a case is relatively more desirable if the auditing does not need to happen frequently. In short, users can flexibly choose the storage/communication tradeoff parameter $s$ for their different system application scenarios.

TABLE 3
Notation of Cryptographic Operations

| | |
|---|---|
| $Hash^t_{\mathbb{G}_1}$ | hash $t$ values into the group $\mathbb{G}_1$. |
| $Mult^t_{\mathbb{G}}$ | $t$ multiplications in group $\mathbb{G}$. |
| $Exp^t_{\mathbb{G}}(\ell)$ | $t$ exponentiations $g^{a_i}$, for $g \in \mathbb{G}$, $\|a_i\| = \ell$. |
| $m\text{-}MultExp^t_{\mathbb{G}}(\ell)$ | $t$ $m$-term exponentiations $\prod_{i=1}^{m} g^{a_i}$. |
| $Pair^t_{\mathbb{G}_1,\mathbb{G}_2}$ | $t$ pairings $e(u_i, g_i)$, where $u_i \in \mathbb{G}_1$, $g_i \in \mathbb{G}_2$. |
| $m\text{-}MultPair^t_{\mathbb{G}_1,\mathbb{G}_2}$ | $t$ $m$-term pairings $\prod_{i=1}^{m} e(u_i, g_i)$. |

On our not-so-powerful workstation, the measurement shows that the user setup phase (i.e., generating authenticators) achieves a throughput of around 9.0 KB/s and 17.2 KB/s when $s = 1$ and $s = 10$, respectively. These results are not very fast due to the expensive modular exponentiation operations for each 20 byte block sector in the authenticator computation. (See [28] for some similar experimental results.) Note that for each data file to be outsourced, such setup phase happens once only. Further, since the authenticator generation on each block is independent, these one-time operations can be easily parallelized by using multithreading technique on the modern multicore systems. Therefore, various optimization techniques can be applied to speedup the user side setup phase. As our paper focuses on privacy-preserving storage auditing performance, in the following, we will primarily assess the performance of the proposed auditing schemes on both TPA side and cloud server side, and show they are indeed lightweight. We will focus on the cost of the privacy-preserving protocol and our proposed batch auditing technique.

#### 4.2.1 Cost of Privacy-Preserving Protocol

We begin by estimating the cost in terms of basic cryptographic operations (refer to Table 3 for notations). Suppose there are $c$ random blocks specified in the challenge message $chal$ during the `Audit` phase. Under this setting, we quantify the cost introduced by the privacy-preserving auditing in terms of server computation, auditor computation as well as communication overhead. Since the difference for choices on $s$ has been discussed previously, in the following privacy-preserving cost analysis we only give the atomic operation analysis for the case $s = 1$ for simplicity. The analysis for the case of $s = 10$ follows similarly and is thus omitted.

On the server side, the generated response includes an aggregated authenticator $\sigma = \prod_{i \in I} \sigma_i^{\nu_i} \in \mathbb{G}_1$, a random factor $R = e(u, v)^r \in \mathbb{G}_T$, and a blinded linear combination

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
Performance under Different Number of Sampled Blocks $c$ for High Assurance ($\geq 95\%$) Auditing

| s = 1 | Our Scheme | | [13] | |
|---|---|---|---|---|
| Sampled blocks $c$ | 460 | 300 | 460 | 300 |
| Sever comp. time (ms) | 335.17 | 219.27 | 333.23 | 217.33 |
| TPA comp. time (ms) | 530.60 | 357.53 | 526.77 | 353.70 |
| Comm. cost (Byte) | 160 | 160 | 40 | 40 |
| s = 10 | Our Scheme | | [13] | |
| Sampled blocks $c$ | 460 | 300 | 460 | 300 |
| Sever comp. time (ms) | 361.56 | 242.29 | 342.91 | 223.64 |
| TPA comp. time (ms) | 547.39 | 374.32 | 543.35 | 370.29 |
| Comm. cost (Byte) | 1420 | 1420 | 220 | 220 |

of sampled blocks $\mu = \gamma \sum_{i \in I} \nu_i m_i + r \in \mathbb{Z}_p$, where $\gamma = h(R) \in \mathbb{Z}_p$. The corresponding computation cost is $c\text{-}MultExp^1_{\mathbb{G}_1}(|\nu_i|)$, $Exp^1_{\mathbb{G}_T}(|p|)$, and $Hash^1_{\mathbb{Z}_p} + Add^c_{\mathbb{Z}_p} + Mult^{c+1}_{\mathbb{Z}_p}$, respectively. Compared to the existing HLA-based solution for ensuring remote data integrity [13], the extra cost resulted from the random mask $R$ is only a constant: $Exp^1_{\mathbb{G}_T}(|p|) + Mult^1_{\mathbb{Z}_p} + Hash^1_{\mathbb{Z}_p} + Add^1_{\mathbb{Z}_p}$, which has nothing to do with the number of sampled blocks $c$. When $c$ is set to be 300 to 460 for high assurance of auditing, as discussed in Section 3.4, the extra cost on the server side for privacy-preserving guarantee would be negligible against the total server computation for response generation.

Similarly, on the auditor side, upon receiving the response $\{\sigma, R, \mu\}$, the corresponding computation cost for response validation is $Hash^1_{\mathbb{Z}_p} + c\text{-}MultExp^1_{\mathbb{G}_1}(|\nu_i|) + Hash^c_{\mathbb{G}_1} + Mult^1_{\mathbb{G}_1} + Mult^1_{\mathbb{G}_T} + Exp^3_{\mathbb{G}_1}(|p|) + Pair^2_{\mathbb{G}_1,\mathbb{G}_2}$, among which only $Hash^1_{\mathbb{Z}_p} + Exp^2_{\mathbb{G}_1}(|p|) + Mult^1_{\mathbb{G}_T}$ account for the additional constant computation cost. For $c = 460$ or 300, and considering the relatively expensive pairing operations, this extra cost imposes little overhead on the overall cost of response validation, and thus can be ignored. For the sake of completeness, Table 4 gives the experiment result on performance comparison between our scheme and the state of the art [13]. It can be shown that the performance of our scheme is almost the same as that of [13], even if our scheme supports privacy-preserving guarantee while [13] does not. For the extra communication cost of our scheme when compared with [13], the server's response $\{\sigma, R, \mu\}$ contains an additional random element $R$, which is a group element of $\mathbb{G}_T$ and has the size close to 960 bits.

#### 4.2.2 Batch Auditing Efficiency

Discussion in Section 3.5 gives an asymptotic efficiency analysis on the batch auditing, by considering only the total number of pairing operations. However, on the practical side, there are additional less expensive operations required for batching, such as modular exponentiations and multiplications. Thus, whether the benefits of removing pairings significantly outweighs these additional operations remains to be verified. To get a complete view of batching efficiency, we conduct a timed batch auditing test, where the number of auditing tasks is increased from 1 to approximately 200 with intervals of 8. Note that we only focus on the choice of $s = 1$ here, from which similar performance results can be directly obtained for the choice of $s = 10$. The performance of the corresponding nonbatched (individual) auditing is provided as a baseline for the measurement. Following the same settings $c = 300$ and $c = 460$, the average per task auditing time, which is computed by dividing total auditing time by the number of tasks, is given in Fig. 2 for both batch and individual auditing. It can be shown that compared to individual auditing, batch auditing indeed helps reducing the TPA's computation cost, as more than 15 percent of per-task auditing time is saved.

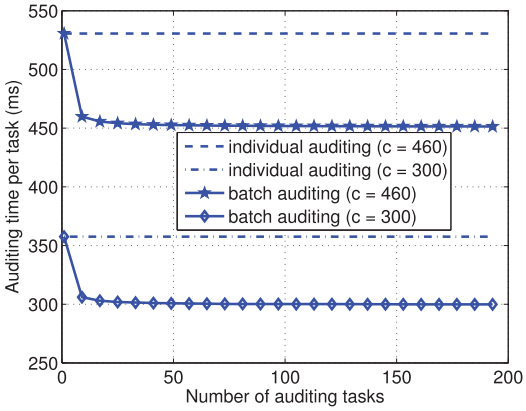



Fig. 2. Comparison on auditing time between batch and individual auditing: Per task auditing time denotes the total auditing time divided by the number of tasks.

#### 4.2.3 Sorting Out Invalid Responses

Now, we use experiment to justify the efficiency of our recursive binary search approach for the TPA to sort out the invalid responses for negative batch auditing result, as discussed in Section 3.5. This experiment is tightly pertained to the work in [20], which evaluates the batch verification of various short signatures.

The feasibility of the recursive approach is evaluated under the choice of $s = 1$, which is consistent with the experiment settings in Section 4.2.2. We do not duplicate evaluation of the recursive binary search methodology for $s = 10$, because similar results can be easily deduced from the choice of $s = 1$. We first generate a collection of 256 valid responses, which implies the TPA may concurrently handle 256 different auditing delegations. We then conduct the tests repeatedly while randomly corrupting an $\alpha$-fraction, ranging from 0 to 20 percent, by replacing them with random values. The average auditing time per task against the individual auditing approach is presented in Fig. 3. The result shows that even when the number of invalid responses exceeds 18 percent of the total batch size, the performance of batch auditing can still be safely concluded as more preferable than the straightforward individual auditing. Note that the random distribution of invalid responses within the collection is nearly the worst case for batch auditing. If invalid responses are grouped together, even better results can be expected.

## 5 Zero Knowledge Public Auditing

Though our scheme prevents the TPA from directly deriving $\mu'$ from $\mu$, it does not rule out the possibility of offline

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

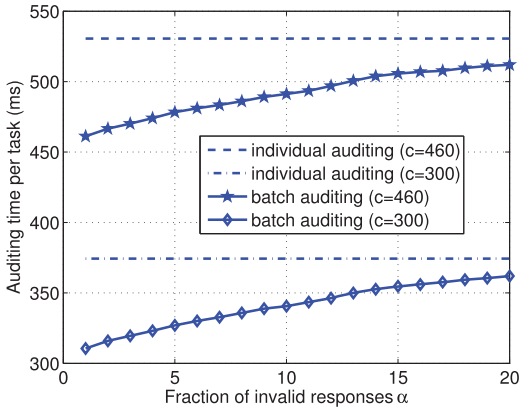



Fig. 3. Comparison on auditing time between batch and individual auditing, when $\alpha$-fraction of 256 responses are invalid: Per task auditing time denotes the total auditing time divided by the number of tasks.

guessing threat by TPA using valid $\sigma$ from the response. Specifically, the TPA can always guess whether $\mu' \stackrel{?}{=} \tilde{\mu}'$, by checking $e(\sigma, g) \stackrel{?}{=} e((\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}) \cdot u^{\tilde{\mu}'}, v)$, where $\tilde{\mu}'$ is constructed from random coefficients chosen by the TPA in the challenge and the guessed message $\{\tilde{m}_i\}_{s_1 \le i \le s_c}$. However, we must note that $\tilde{\mu}'$ is chosen from $\mathbb{Z}_p$ and $|p|$ is usually larger than 160 bits in practical security settings (see Section 4.2). Given no background information, the success of this all-or-nothing guess on $\mu'$ launched by TPA over such a large space $\mathbb{Z}_p$ can be very difficult. Besides, because TPA must at least make $c$ successful guesses on the same set of blocks to derive $\{m_i\}_{s_1 \le i \le s_c}$ from the system of $c$ linear equations, we can specify $c$ to be large enough in the protocol (e.g., as discussed in Section 3.4, a strict choice of $c$ should be at least larger than 460), which can significantly decrease the TPA's successful guessing probability. In addition, we can also restrict the number of reauditing on exactly the same set of blocks (e.g., to limit the repeated auditing times on exactly the same set of blocks to be always less than $c$). In this way, TPA can be kept from accumulating successful guesses on $\mu'$ for the same set of blocks, which further diminishes the chance for TPA to solve for $\{m_i\}_{s_1 \le i \le s_c}$. In short, by appropriate choices of parameter $c$ and group size $\mathbb{Z}_p$, we can effectively defeat such potential offline guessing threat.

Nevertheless, we present a public auditing scheme with provably zero knowledge leakage. This scheme can completely eliminate the possibilities of above offline guessing attack, but at the cost of a little higher communication and computation overhead. The `setup` phase is similar to our main scheme presented in Section 3.4. The secret parameters are $sk = (x, ssk)$ and the public parameters are $pk = (spk, v, g, u, e(u, v), g_1)$, where $g_1 \in \mathbb{G}_1$ is an additional public group element. In the `audit` phase, upon receiving challenge $chal = \{(i, \nu_i)\}_{i \in I}$, the server chooses three random elements $r_m, r_\sigma, \rho \leftarrow \mathbb{Z}_p$, and calculates $R = e(g_1, g)^{r_\sigma} \cdot e(u, v)^{r_m} \in \mathbb{G}_T$ and $\gamma = h(R) \in \mathbb{Z}_p$. Let $\mu'$ denote the linear combination of sampled blocks $\mu' = \sum_{i \in I} \nu_i m_i$, and $\sigma$ denote the aggregated authenticator $\sigma = \prod_{i \in I} \sigma_i^{\nu_i} \in \mathbb{G}_1$. To ensure the auditing leaks zero knowledge, the server has to blind both $\mu'$ and $\sigma$. Specifically, the server computes: $\mu = r_m + \gamma \mu' \bmod p$, and $\Sigma = \sigma \cdot g_1^{\rho}$. It then sends $\{\varsigma, \mu, \Sigma, R\}$ as the response proof of storage correctness to the TPA, where $\varsigma = r_\sigma + \gamma\rho \bmod p$. With the response from the server, the TPA runs `VerifyProof` to validate the response by first computing $\gamma = h(R)$ and then checking the verification equation

$$R \cdot e(\Sigma^{\gamma}, g) \stackrel{?}{=} e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{\gamma} \cdot u^{\mu}, v\right) \cdot e(g_1, g)^{\varsigma}. \quad (6)$$

To see the correctness of the above equation, we have

$$\begin{aligned} R \cdot e(\Sigma^{\gamma}, g) &= e(g_1, g)^{r_\sigma} \cdot e(u, v)^{r_m} \cdot e((\sigma \cdot g_1^{\rho})^{\gamma}, g) \\ &= e(g_1, g)^{r_\sigma} \cdot e(u, v)^{r_m} \cdot e((\sigma^{\gamma}, g) \cdot e(g_1^{\rho\gamma}, g) \\ &= e(u, v)^{r_m} \cdot e((\sigma^{\gamma}, g) \cdot e(g_1, g)^{r_\sigma + \rho\gamma} \\ &= e\left(\left(\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i}\right)^{\gamma} \cdot u^{\mu}, v\right) \cdot e(g_1, g)^{\varsigma}. \end{aligned}$$

The last equality follows from the elaboration of (1) in Section 3.4.

**Theorem 4.** *The above auditing protocol achieves zero-knowledge information leakage to the TPA, and it also ensures the storage correctness guarantee.*

**Proof.** Zero-knowledge is easy to see. Randomly pick $\gamma, \mu, \varsigma$ from $\mathbb{Z}_p$ and $\Sigma$ from $\mathbb{G}_1$, set $R \leftarrow e((\prod_{i=s_1}^{s_c} H(W_i)^{\nu_i})^{\gamma} \cdot u^{\mu}, v) \cdot e(g_1, g)^{\varsigma} / e(\Sigma^{\gamma}, g)$ and backpatch $\gamma = h(R)$. For proof of storage correctness, we can extract $\rho$ similar to the extraction of $\mu'$ as in the proof of Theorem 1. Likewise, $\sigma$ can be recovered from $\Sigma$. To conclude, a valid pair of $\sigma$ and $\mu'$ can be extracted. □

## 6 Related Work

Ateniese et al. [9] are the first to consider public auditability in their "provable data possession" (PDP) model for ensuring possession of data files on untrusted storages. They utilize the RSA-based homomorphic linear authenticators for auditing outsourced data and suggest randomly sampling a few blocks of the file. However, among their two proposed schemes, the one with public auditability exposes the linear combination of sampled blocks to external auditor. When used directly, their protocol is not provably privacy preserving, and thus may leak user data information to the external auditor. Juels et al. [11] describe a "proof of retrievability" (PoR) model, where spot-checking and error-correcting codes are used to ensure both "possession" and "retrievability" of data files on remote archive service systems. However, the number of audit challenges a user can perform is fixed a priori, and public auditability is not supported in their main scheme. Although they describe a straightforward Merkle-tree construction for public PoRs, this approach only works with encrypted data. Later, Bowers et al. [18] propose an improved framework for POR protocols that generalizes Juels' work. Dodis et al. [29] also give a study on different variants of PoR with private auditability. Shacham and Waters [13] design an improved PoR scheme built from BLS signatures [19] with proofs of security in the security model defined in [11]. Similar to the construction in [9], they use publicly verifiable homomorphic linear authenticators that are built from provably secure BLS signatures. Based on

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

the elegant BLS construction, a compact and public verifiable scheme is obtained. Again, their approach is not privacy preserving due to the same reason as [9]. Shah et al. [15], [10] propose introducing a TPA to keep online storage honest by first encrypting the data then sending a number of pre-computed symmetric-keyed hashes over the encrypted data to the auditor. The auditor verifies the integrity of the data file and the server's possession of a previously committed decryption key. This scheme only works for encrypted files, requires the auditor to maintain state, and suffers from bounded usage, which potentially brings in online burden to users when the keyed hashes are used up.

Dynamic data have also attracted attentions in the recent literature on efficiently providing the integrity guarantee of remotely stored data. Ateniese et al. [21] is the first to propose a partially dynamic version of the prior PDP scheme, using only symmetric key cryptography but with a bounded number of audits. In [22], Wang et al. consider a similar support for partially dynamic data storage in a distributed scenario with additional feature of data error localization. In a subsequent work, Wang et al. [8] propose to combine BLS-based HLA with MHT to support fully data dynamics. Concurently, Erway et al. [23] develop a skip list-based scheme to also enable provable data possession with full dynamics support. However, the verification in both protocols requires the linear combination of sampled blocks as an input, like the designs in [9], [13], and thus does not support privacy-preserving auditing.

In other related work, Sebe et al. [30] thoroughly study a set of requirements which ought to be satisfied for a remote data possession checking protocol to be of practical use. Their proposed protocol supports unlimited times of file integrity verifications and allows preset tradeoff between the protocol running time and the local storage burden at the user. Schwarz and Miller [31] propose the first study of checking the integrity of the remotely stored data across multiple distributed servers. Their approach is based on erasure-correcting code and efficient algebraic signatures, which also have the similar aggregation property as the homomorphic authenticator utilized in our approach. Curtmola et al. [32] aim to ensure data possession of multiple replicas across the distributed storage system. They extend the PDP scheme in [9] to cover multiple replicas without encoding each replica separately, providing guarantee that multiple copies of data are actually maintained. In [33], Bowers et al. utilize a two-layer erasure-correcting code structure on the remotely archived data and extend their POR model [18] to distributed scenario with high-data availability assurance. While all the above schemes provide methods for efficient auditing and provable assurance on the correctness of remotely stored data, almost none of them necessarily meet all the requirements for privacy-preserving public auditing of storage. Moreover, none of these schemes consider batch auditing, while our scheme can greatly reduce the computation cost on the TPA when coping with a large number of audit delegations.

Portions of the work presented in this paper have previously appeared as an extended abstract in [1]. We have revised the paper a lot and improved many technical details as compared to [1]. The primary improvements are as follows: First, we provide a new privacy-preserving public auditing protocol with enhanced security strength in Section 3.4. For completeness, we also include an additional (but slightly less efficient) protocol design for provably secure zero-knowledge leakage public auditing scheme in Section 5. Second, based on the enhanced main auditing scheme, we provide a new provably secure batch auditing protocol. All the experiments in our performance evaluation for the newly designed protocol are completely redone. Third, we extend our main scheme to support data dynamics in Section 3.6, and provide discussions on how to generalize our privacy-preserving public auditing scheme in Section 3.7, which are lacking in [1]. Finally, we provide formal analysis of privacy-preserving guarantee and storage correctness, while only heuristic arguments are sketched in [1].

## 7 Conclusion

In this paper, we propose a privacy-preserving public auditing system for data storage security in cloud computing. We utilize the homomorphic linear authenticator and random masking to guarantee that the TPA would not learn any knowledge about the data content stored on the cloud server during the efficient auditing process, which not only eliminates the burden of cloud user from the tedious and possibly expensive auditing task, but also alleviates the users' fear of their outsourced data leakage. Considering TPA may concurrently handle multiple audit sessions from different users for their outsourced data files, we further extend our privacy-preserving public auditing protocol into a multiuser setting, where the TPA can perform multiple auditing tasks in a batch manner for better efficiency. Extensive analysis shows that our schemes are provably secure and highly efficient. Our preliminary experiment conducted on Amazon EC2 instance further demonstrates the fast performance of our design on both the cloud and the auditor side. We leave the full-fledged implementation of the mechanism on commercial public cloud as an important future extension, which is expected to robustly cope with very large scale data and thus encourage users to adopt cloud storage services more confidently.

## Acknowledgments

This work was supported in part by the US National Science Foundation (NSF) under grants CNS-1054317, CNS-1116939, CNS-1156318, and CNS-1117111, and by Amazon web service research grant. A preliminary version [1] of this paper was presented at the 29th IEEE Conference on Computer Communications (INFOCOM '10).

## References

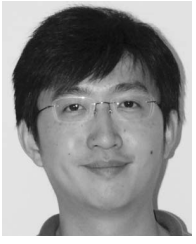
[1] C. Wang, Q. Wang, K. Ren, and W. Lou, "Privacy-Preserving Public Auditing for Storage Security in Cloud Computing," *Proc. IEEE INFOCOM '10*, Mar. 2010.

[2] P. Mell and T. Grance, "Draft NIST Working Definition of Cloud Computing," http://csrc.nist.gov/groups/SNS/cloud-computing/index.html, June 2009.

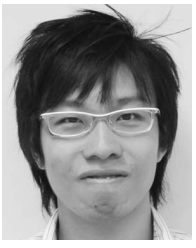
[3] M. Armbrust, A. Fox, R. Griffith, A.D. Joseph, R.H. Katz, A. Konwinski, G. Lee, D.A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "Above the Clouds: A Berkeley View of Cloud Computing," Technical Report UCB-EECS-2009-28, Univ. of California, Berkeley, Feb. 2009.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.

[4] Cloud Security Alliance, "Top Threats to Cloud Computing," http://www.cloudsecurityalliance.org, 2010.
[5] M. Arrington, "Gmail Disaster: Reports of Mass Email Deletions," http://www.techcrunch.com/2006/12/28/gmail-disasterreports-of-mass-email-deletions/, 2006.
[6] J. Kincaid, "MediaMax/TheLinkup Closes Its Doors," http://www.techcrunch.com/2008/07/10/mediamaxthelinkup-closes-its-doors/, July 2008.
[7] Amazon.com, "Amazon s3 Availability Event: July 20, 2008," http://status.aws.amazon.com/s3-20080720.html, July 2008.
[8] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, "Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing," *IEEE Trans. Parallel and Distributed Systems,* vol. 22, no. 5, pp. 847-859, May 2011.
[9] G. Ateniese, R. Burns, R. Curtmola, J. Herring, L. Kissner, Z. Peterson, and D. Song, "Provable Data Possession at Untrusted Stores," *Proc. 14th ACM Conf. Computer and Comm. Security (CCS '07),* pp. 598-609, 2007.
[10] M.A. Shah, R. Swaminathan, and M. Baker, "Privacy-Preserving Audit and Extraction of Digital Contents," Cryptology ePrint Archive, Report 2008/186, 2008.
[11] A. Juels and J. Burton, S. Kaliski, "PORs: Proofs of Retrievability for Large Files," *Proc. ACM Conf. Computer and Comm. Security (CCS '07),* pp. 584-597, Oct. 2007.
[12] Cloud Security Alliance, "Security Guidance for Critical Areas of Focus in Cloud Computing," http://www.cloudsecurityalliance.org, 2009.
[13] H. Shacham and B. Waters, "Compact Proofs of Retrievability," *Proc. Int'l Conf. Theory and Application of Cryptology and Information Security: Advances in Cryptology (Asiacrypt),* vol. 5350, pp. 90-107, Dec. 2008.
[14] C. Wang, K. Ren, W. Lou, and J. Li, "Towards Publicly Auditable Secure Cloud Data Storage Services," *IEEE Network Magazine,* vol. 24, no. 4, pp. 19-24, July/Aug. 2010.
[15] M.A. Shah, M. Baker, J.C. Mogul, and R. Swaminathan, "Auditing to Keep Online Storage Services Honest," *Proc. 11th USENIX Workshop Hot Topics in Operating Systems (HotOS '07),* pp. 1-6, 2007.
[16] 104th United States Congress, "Health Insurance Portability and Accountability Act of 1996 (HIPPA)," http://aspe.hhs.gov/admnsimp/pl104191.htm, 1996.
[17] R. Curtmola, O. Khan, and R. Burns, "Robust Remote Data Checking," *Proc. Fourth ACM Int'l Workshop Storage Security and Survivability (StorageSS '08),* pp. 63-68, 2008.
[18] K.D. Bowers, A. Juels, and A. Oprea, "Proofs of Retrievability: Theory and Implementation," *Proc. ACM Workshop Cloud Computing Security (CCSW '09),* pp. 43-54, 2009.
[19] D. Boneh, B. Lynn, and H. Shacham, "Short Signatures from the Weil Pairing," *J. Cryptology,* vol. 17, no. 4, pp. 297-319, 2004.
[20] A.L. Ferrara, M. Green, S. Hohenberger, and M. Pedersen, "Practical Short Signature Batch Verification," *Proc. Cryptographers' Track at the RSA Conf. 2009 on Topics in Cryptology (CT-RSA),* pp. 309-324, 2009.
[21] G. Ateniese, R.D. Pietro, L.V. Mancini, and G. Tsudik, "Scalable and Efficient Provable Data Possession," *Proc. Int'l Conf. Security and Privacy in Comm. Networks (SecureComm '08),* pp. 1-10, 2008.
[22] C. Wang, Q. Wang, K. Ren, and W. Lou, "Towards Secure and Dependable Storage Services in Cloud Computing," *IEEE Trans. Service Computing,* vol. 5, no. 2, 220-232, Apr.-June 2012.
[23] C. Erway, A. Kupcu, C. Papamanthou, and R. Tamassia, "Dynamic Provable Data Possession," *Proc. ACM Conf. Computer and Comm. Security (CCS '09),* pp. 213-222, 2009.
[24] R.C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. IEEE Symp. Security and Privacy,* 1980.
[25] G. Ateniese, S. Kamara, and J. Katz, "Proofs of Storage from Homomorphic Identification Protocols," *Proc. 15th Int'l Conf. Theory and Application of Cryptology and Information Security: Advances in Cryptology (ASIACRYPT),* pp. 319-333, 2009.
[26] M. Bellare and G. Neven, "Multi-Signatures in the Plain Public-Key Model and a General Forking Lemma," *Proc. ACM Conf. Computer and Comm. Security (CCS),* pp. 390-399, 2006.
[27] Amazon.com, "Amazon Elastic Compute Cloud," http://aws.amazon.com/ec2/, 2009.
[28] Y. Zhu, H. Wang, Z. Hu, G.-J. Ahn, H. Hu, and S. Yau, "Efficient Provable Data Possession for Hybrid Clouds," Cryptology ePrint Archive, Report 2010/234, 2010.
[29] Y. Dodis, S.P. Vadhan, and D. Wichs, "Proofs of Retrievability via Hardness Amplification," *Proc. Theory of Cryptography Conf. Theory of Cryptography (TCC),* pp. 109-127, 2009.
[30] F. Sebe, J. Domingo-Ferrer, A. Martínez-Balleste, Y. Deswarte, and J.-J. Quisquater, "Efficient Remote Data Possession Checking in Critical Information Infrastructures," *IEEE Trans. Knowledge and Data Eng.,* vol. 20, no. 8, pp. 1034-1038, Aug. 2008.
[31] T. Schwarz and E.L. Miller, "Store, Forget, and Check: Using Algebraic Signatures to Check Remotely Administered Storage," *Proc. IEEE Int'l Conf. Distributed Computing Systems (ICDCS '06),* 2006.
[32] R. Curtmola, O. Khan, R. Burns, and G. Ateniese, "MR-PDP: Multiple-Replica Provable Data Possession," *Proc. IEEE Int'l Conf. Distributed Computing Systems (ICDCS '08),* pp. 411-420, 2008.
[33] K.D. Bowers, A. Juels, and A. Oprea, "HAIL: A High-Availability and Integrity Layer for Cloud Storage," *Proc. ACM Conf. Computer and Comm. Security (CCS '09),* pp. 187-198, 2009.



**Cong Wang** is an assistant professor in the Computer Science Department at City University of Hong Kong. He received his BE and ME degrees from Wuhan University in 2004 and 2007, and the PhD degree from Illinois Institute of Technology in 2012, all in electrical and computer engineering. He worked at the Palo Alto Research Center in the summer of 2011. His research interests are in the areas of cloud computing security, with current focus on secure data outsourcing and secure computation outsourcing in public cloud. He is a member of the ACM and a member of the IEEE.



**Sherman S.M. Chow** received the BEng (first class honors) and MPhil degrees from the University of Hong Kong, and the MS and PhD degrees from Courant Institute of Mathematical Sciences, New York University. He is an assistant professor in the Department of Information Engineering at the Chinese University of Hong Kong, a position he commenced after his post-doctoral research at the Department of Combinatorics and Optimization, University of Waterloo. He interned at NTT Research and Development (Tokyo), Microsoft Research (Redmond) and Fuji Xerox Palo Alto Laboratory, and has made research visits to the University of Maryland, University of Calgary, University of Texas, HKU, MIT, and Queensland University of Technology. His research interests are applied cryptography, privacy and distributed systems security in general. He serves on the program committees of several international conferences including ACNS 2012-2013, ASIACRYPT 2012-2013, WPES 2012, ASIACCS 2013, IEEE-CNS 2013 and Financial Crypt. 2013



**Qian Wang** received the BS degree from Wuhan University, China, in 2003, his MS degree from Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Sciences, China, in 2006, and his PhD degree from Illinois Institute of Technology in 2012, all in electrical and computer engineering. Currently, he is an associate professor with the School of Computer, Wuhan University. His research interests include wireless network security and privacy, cloud computing security, and applied cryptography. He is a corecipient of the Best Paper Award from IEEE ICNP 2011. He is a member of the ACM and a member of the IEEE.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.



**Kui Ren** is an associate professor in the Computer Science and Engineering Department at the State University of New York at Buffalo. Prior to that, he was an assistant/associate professor in the Electrical and Computer Engineering Department at Illinois Institute of Technology. He obtained his PhD degree in electrical and computer engineering from Worcester Polytechnic Institute in 2007. His research interests include security and privacy in cloud computing, wireless security, and smart grid security. His research is supported by NSF, DOE, AFRL, and Amazon. He was a recipient of an NSF Faculty Early Career Development (CAREER) Award in 2011 and a corecipient of the Best Paper Award of IEEE ICNP 2011. He serves on the editorial boards of *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Smart Grid*, *IEEE Wireless Communications*, *IEEE Communications Surveys and Tutorials*, and *Journal of Communications and Networks*. He is a senior member of the IEEE.



**Wenjing Lou** is an associate professor at Virginia Polytechnic Institute and State University. Prior to joining Virginia Tech in 2011, she was on the faculty of Worcester Polytechnic Institute from 2003 to 2011. She received her PhD in Electrical and Computer Engineering at the University of Florida in 2003. Her current research interests are in cyber security, with emphases on wireless network security and data security and privacy in cloud computing. She was a recipient of the US National Science Foundation CAREER award in 2008. She is a senior member of the IEEE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 03:11:20 UTC from IEEE Xplore. Restrictions apply.